

PAT1:-

Problem Statement

Evelyn is a software developer working on a system that calculates bonuses for employees based on their performance and years of service. The bonus is determined by a specific set of rules. Your task is to help Evelyn implement a program that calculates the bonus amount based on the given rules.

Bonus Calculation:

Performance Rating 1 or 2: No bonus, regardless of years of service.

Performance Rating 3:

If years of service are less than 5, the bonus is 500 rupees.

If years of service are 5 or more, the bonus is 1000 rupees.

Performance Rating 4:

If years of service are less than 3, the bonus is 1500 rupees.

If years of service are 3 or more, the bonus is 2000 rupees.

Performance Rating 5:

If years of service are less than 2, the bonus is 2500 rupees.

If years of service are 2 or more, the bonus is 3000 rupees.

Input format :

The first line of input consists of an integer P, representing the performance rating.

The second line consists of an integer Y, representing the years of service.

Output format :

The output displays an integer representing the bonus amount.

Refer to the sample output for formatting specifications.

Code constraints :

$$1 \leq P \leq 5$$

$$0 \leq Y \leq 30$$

Sample test cases :

Input 1 :

3

4

Output 1 :

500

Input 2 :

5

3

Output 2 :

3000

Problem Statement

Liam is organizing a community event and allocates 30% of his total funds for decorations and 20% for refreshments. Write a program to calculate the remaining amount after these expenses. Input the total funds and output the remaining amount formatted to two decimal places.

Input format :

The input consists of a double value B, representing the total funds for the

event.

Output format :

The output displays a double value, representing the remaining amount rounded to two decimal places.

Refer to the sample output for formatting specifications.

Code constraints :

$1000.00 \leq B \leq 10000.00$

Sample test cases :

Input 1 :

1000.00

Output 1 :

500.00

Input 2 :

4504.20

Output 2 :

2252.10

PAT2:-

Problem Statement

Hannah is analyzing the performance of a network by examining packet transmission data. She needs to count the number of packets that were retransmitted and determine the packet with the highest transmission time.

Write a program to help Hannah with this analysis.

Input format :

The first line of input consists of an integer N, representing the number of packets.

The next 2n lines each contain an integer:

The first line contains an integer T, representing the transmission time of the packet and

The second line contains an integer, representing its retransmission status (1 for retransmitted, 0 for not).

Output format :

The first line of output displays an integer representing the number of retransmitted packets.

The second line of output displays an integer representing the highest transmission time.

Refer to the sample output for formatting specifications.

Code constraints :

$1 \leq N \leq 20$

$1 \leq T \leq 1000$

Sample test cases :

Input 1 :

3

100

1

200

0

150

1

Output 1 :

2
200

Input 2 :

2
80
0
250

Output 2 :

1
250

Single File Programming Question Problem Statement

Rina is managing the inventory for a library, where each row of a 2D matrix represents the number of different genres of books available on each shelf. To get a better overview, she wants to replace each number in a row with the sum of all numbers in that row.

Write a program to help Rina transform the matrix accordingly.

Input format :

The first line of input consists of an integer T, representing the number of test cases.

For each test case:

The first line contains two integers R and C, representing the number of rows and columns.

The next R lines each contain C space-separated integers, representing the count of books of a specific genre on a shelf.

Output format :

For each test case, the output prints the transformed 2D matrix where each element in a row is replaced with the sum of the elements in that row.

Refer to the sample output for formatting specifications.

Code constraints :

The given test cases fall under the following constraints:

$1 \leq T \leq 10$

$1 \leq R, C \leq 10$

$0 \leq \text{books count} \leq 100$

Sample test cases :

Input 1 :

2
3 4
8 2 4 9
4 5 6 1
7 8 9 3
2 5
1 2 3 5 7
3 4 8 9 6

Output 1 :

23 23 23 23
16 16 16 16
27 27 27 27
18 18 18 18 18
30 30 30 30 30

```
Input 2 :
1
3 3
48 65 83
45 21 76
71 32 98
Output 2 :
196 196 196
142 142 142
201 201 201
```

PAT3:-

Question No: 1

Single File Programming Question
Problem Statement

Sarah is training for a marathon and tracks her workouts using a fitness app. She monitors the duration of each exercise session and calculates calories burned per minute. Her goal is to reach a specific total calorie burn, and she needs to know the total calories burned after each session and whether she has achieved her goal.

Your task is to write a program using the `FitnessTracker` class to calculate the total calories burned, the percentage of the goal achieved, and determine if the goal has been reached.

Formula:

Total Calories Burned=duration×caloriesBurnedPerMinute

Percentage achieved=totalCaloriesBurned / fitnessGoal * 100

Note: Use the Parameterized methods.

Input format :

The first line of input consists of an integer D, representing the duration of the workout session in minutes.

The second line of input consists of an integer C, representing the calories burned per minute during the workout.

The third line of input consists of an integer F, representing Sarah's target calories to burn.

Output format :

The first line of output prints "Total calories burned: <totalCaloriesBurned>", where <totalCaloriesBurned> is the total calories burned after the exercise session.

The second line of output prints "Fitness goal: <fitnessGoal>", where <fitnessGoal> is the total calorie goal.

The third line of output prints "Percentage of goal achieved: <percentage>", where <percentage> is the percentage of the goal achieved, rounded to two decimal places.

If the total calories burned is greater than or equal to the fitness goal, print "Goal reached!".

Otherwise, print "Keep going!"

Refer to the sample output for the formatting specifications.

Code constraints :

$1 \leq D \leq 120$

$1 \leq C \leq 10$

$1 \leq F \leq 600$

Sample test cases :

Input 1 :

30

10

300

Output 1 :

Total calories burned: 300

Fitness goal: 300

Percentage of goal achieved: 100.00%

Goal reached!

Input 2 :

20

8

250

Output 2 :

Total calories burned: 160

Fitness goal: 250

Percentage of goal achieved: 64.00%

Keep going!

Single File Programming Question

Problem Statement

In a company called "Tech Innovators," the management wants to calculate the total compensation for its employees, including their salary and bonus. You are tasked with creating a Java program to help with this calculation. You need to implement a class named Employee that contains the following:

A parameterized constructor to initialize the employee's salary and bonus (both are double values).

A method getTotalCompensation() that calculates the total compensation using the formula:

Total compensation = salary + bonus

A static method displayCompensationStatistics(Employee[] employees) that takes an array of Employee objects and displays the highest and lowest total compensation.

The program should create an array of Employee objects to represent multiple employees and calculate their respective total compensations.

Input format :

The first line gives the integer n, representing the number of employees.

The following lines contain alternating salary and bonus values for each employee.

Output format :

The first line of output displays a double value representing the highest total compensation formatted to two decimal places.

The second line of output displays a double value representing the lowest total compensation formatted to two decimal places.

Refer to the sample output for formatting specifications.

Code constraints :

The given test cases fall under the following constraints:

$1 \leq n \leq 100$

$0.0 \leq \text{salary, bonus} \leq 1,000,000.0$

Sample test cases :

Input 1 :

3
10000
1000
20000
2000
30000
3000

Output 1 :

33000.00
11000.00

Input 2 :

4
20000
2000
22000
2200
24000
2400
26000
2600

Output 2 :

28600.00
22000.00

PAT4:-

Question No: 1

Single File Programming Question

Problem Statement

You need to develop a system simulating different cryptocurrency wallets and calculating transaction fees and total amounts after transactions. Implement an abstract Cryptowallet class with methods to compute transaction fees, total amount after fees, and yearly fees.

Subclasses include:

BitcoinWallet: 1% (0.01) transaction fee

EthereumWallet: 1.5% t(0.015) transaction fee

AltcoinWallet: 2% (0.02) transaction fee, with additional methods for discounted fees(10%).

Formulas used:

Transaction Fee = Amount $\times$ Fee Percentage

Total Amount After Fee = Amount - Transaction Fee

Yearly Transaction Fees = Transaction Fee $\times$ 12

Total Amount After Discounted Fee (AltcoinWallet) = Amount - (Transaction Fee $\times$ (1 - Discount Percentage))

Input format :

The first line contains an integer walletType representing the type of wallet:

1 for Bitcoin Wallet.
2 for Ethereum Wallet.
3 for Altcoin Wallet.

The second line contains a double amount, representing the amount of cryptocurrency in dollars.

Output format :

For valid wallet types (walletType = 1, 2, or 3):

The first line contains the transaction fee: Fee: X.XX.

The second line contains the total amount after the fee: Total after fee: X.XX.

The third line contains the yearly transaction fees: Yearly fees: X.XX.

For Altcoin Wallet (walletType = 3), additionally:

The fourth line contains the total amount after applying a discount of 10% to the fee: Total after discount: X.XX.

If the input wallet type is invalid (i.e., not 1, 2, or 3), print: Invalid wallet type

Note: All results are formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Code constraints :

$1.0 \leq \text{amount} \leq 10000.0$

Wallet type Method Choice: 1, 2, or 3.

Sample test cases :

Input 1 :

1

1000.00

Output 1 :

Fee: 10.00

Total after fee: 990.00

Yearly fees: 120.00

Input 2 :

2

500.00

Output 2 :

Fee: 7.50

Total after fee: 492.50

Yearly fees: 90.00

Input 3 :

3

1500.00

Output 3 :

Fee: 30.00

Total after fee: 1470.00

Yearly fees: 360.00

Total after discount: 1473.00

Input 4 :

5

20.0

Output 4 :

Invalid wallet type

Question No: 2

Single File Programming Question

Problem Statement

Priya is developing a system for her grocery store to compute the final price and bonus for items. The store sells Vegetables, Fruits, and Dairy, each with specific discount rates and bonus calculations.

Implement a class `GroceryItem` with methods to `calculateDiscountedPrice()` and `calculateBonus()`. The subclasses—`Vegetables`, `Fruits`, and `Dairy`—should override these methods with their respective discount rates (10%, 15%, 5% of the price of the item) and bonuses (2%, 3%, 1% of the price of the item), respectively.

The program will prompt the user to input the item's price and type (1 for Vegetables, 2 for Fruits, 3 for Dairy). It should calculate and display the final price and bonus or output "Invalid item type" for incorrect inputs.

Input format :

The first line consists of a double value representing the price of the item.

The second line consists of an integer representing the type of item (1 for Vegetables, 2 for Fruits, 3 for Dairy).

Output format :

The first line displays the "Final price after discount: " a double, representing the final price after applying the discount, rounded to two decimal places.

The second line displays the "Bonus value: " a double, representing the bonus value, rounded to two decimal places.

If the inputs an item type that is not 1, 2, or 3, it prints: "Invalid item type".

Refer to the sample output for the formatting specifications.

Code constraints :

$0.00 < \text{price} \leq 10000.00$

The item type must be 1, 2, or 3.

Sample test cases :

Input 1 :

100.00

1

Output 1 :

Final price after discount: 90.00

Bonus value: 2.00

Input 2 :

50.00

2

Output 2 :

Final price after discount: 42.50

Bonus value: 1.50

Input 3 :

200.00

3

Output 3 :

Final price after discount: 190.00

Bonus value: 2.00

Input 4 :

45.00

5

Output 4 :

Invalid item type

PAT5:-

Single File Programming Question Problem Statement

Elsa is developing a Java application to handle user input, where each word in a given phrase must be formatted with the first letter capitalized and the remaining letters in lowercase. Create a program using the java.lang package that includes a method to accept a phrase and return it in the desired format. Make sure the output preserves the correct spacing between words.

Input format :

The input consists of a string. The string may consist of multiple words separated by spaces.

Output format :

The output prints the formatted string, where the first letter of each word is capitalized, and all other letters are in lowercase.

Refer to the sample output for formatting specifications.

Code constraints :

The maximum length of the input string is 100.

Sample test cases :

Input 1 :

vit chennai

Output 1 :

Vit Chennai

Input 2 :

java programming language

Output 2 :

Java Programming Language

Input 3 :

SUPER HERO

Output 3 :

Super Hero

Input 4 :

jOhn doE

Output 4 :

John Doe

Question No: 2

Single File Programming Question Problem Statement

Jesan wants to keep track of his Grade Point Averages (GPA) and credit hours for his courses. He needs a program that validates his GPA entries and ensures he does not exceed a total of 120 credit hours.

InvalidGPAEntryException: The GPA must be between 0.0 and 4.0. If it's outside this range, throw an InvalidGPAEntryException with the message "Invalid GPA entry".

TotalCreditHoursExceededException: The total credit hours must not exceed 120. If they do, throw a TotalCreditHoursExceededException with the message "Total credit hours exceeded".

Note: Cumulative GPA = Total Weighted GPA/ Total Credit Hours, where Total Weighted GPA is the sum of each GPA multiplied by its corresponding credit hours, and Total Credit Hours is the sum of each credit hour.

Write a Java program that implements this functionality.

Input format :

The program will accept multiple lines of input until the user types "exit".

Each line should contain the GPA and credit hours separated by a space.

Output format :

For each valid entry, if the GPA is valid and credit hours do not exceed the limit, print the cumulative GPA, rounded off to two decimal places.

If the input is invalid, the program should print an appropriate error message:

Invalid GPA: If the GPA is outside the range of 0.0 to 4.0, print: "Invalid GPA entry".

Total Credit Hours Exceeded: If the total credit hours exceed 120, print: "Total credit hours exceeded".

Refer to the sample output for formatting specifications.

Sample test cases :

Input 1 :

3.5 15

3.8 12

exit

Output 1 :

Cumulative GPA: 3.63

Input 2 :

2.3 23

4.6 13

exit

Output 2 :

Invalid GPA entry

Input 3 :

-3.5 23

exit

Output 3 :

Invalid GPA entry

Input 4 :

3.0 90

2.5 40

exit

Output 4 :

Total credit hours exceeded

Input 5 :

3.2 21

3.7 25

3.9 28

3.5 22

exit

Output 5 :

Cumulative GPA: 3.60